

RUNBOOK — MLOps on Snowflake Demo (Presenter Guide)

The story you're telling: how a data-science team runs a **dev ML sandbox inside a production Snowflake account** — reading production data, iterating on features and models with experiment tracking, and promoting to production through a **keyless, gated CI/CD pipeline** where a data scientist can *never* deploy to prod directly. Fraud / AML is the relatable backdrop; the subject is **Snowflake ML Ops**.

Audience framing: generic financial-services fraud/AML monitoring. Do not name the customer in anything on screen.

0. Screen setup (three screens)

Screen	Purpose
Snowsight Workspace (git-linked to the repo)	Primary. Open the dev notebook; browse Feature Store, Experiments, Model Registry, and the PREDICTIONS table in Snowsight.
Cortex Code (desktop)	Show this build conversation + run the pre-built setup / live retrain / kickoff / reset from a local terminal.
GitHub (repo + Actions)	Trigger the promotion workflow and approve the production environment (the human gate).

Where things run — legend used below:

- **[TERM]** = local terminal in Cortex Code (.venv + snow/python). Runs the .py scripts.
- **[WS]** = Snowsight Workspace notebook (git-linked repo).
- **[UI]** = Snowsight browse (AI & ML / Data).
- **[GH]** = GitHub (Actions tab).

All [TERM] commands assume:

```
cd "RT Feature Store - Fraud Detection"
export SNOWFLAKE_CONNECTION_NAME=<your-connection>
```

During the live demo you never touch a terminal or Cortex Code. Every live step is [WS] (Workspace notebook), [UI] (Snowsight), or [GH] (GitHub). [TERM] is used only for **pre-demo kickoff** and **post-demo reset** — off-stage. (Cortex Code is up only to *show* this build conversation.)

1. Pre-demo kickoff (run ~5 minutes before)

The only thing we can't speed up live is the ML Job's compute-pool cold start. Pre-warm it:

[TERM]

```
ML_ENV=dev .venv/bin/python pre_demo/kickoff.py          # resume + warm pool, readiness check
# optional, if the container image may be cold:
ML_ENV=dev .venv/bin/python pre_demo/kickoff.py --warm-image
```

Expect: pool state=ACTIVE/IDLE nodes_ready=1, dev ABT ~5,077,237 rows, dev model versions ['V1', 'V2'], and a **GO**. Now the promotion job starts without a multi-minute cold start.

Two separate GitHub connections — don't conflate them:

1. **Workspace ↔ GitHub (git link)**: lets Snowsight pull the repo. Configured on the Snowflake side with an API integration + a **Snowflake SECRET holding a GitHub PAT**. This is your connectivity task; it has nothing to do with the pipeline.
2. **GitHub Actions → Snowflake (promotion pipeline): keyless (OIDC/WIF)** — no key, no password, no credential secret. The only value it needs is SNOWFLAKE_ACCOUNT, the account identifier, stored as a GitHub **variable** (not a credential). Plus the production environment for the approval gate.

2. What's pre-built vs. live

Pre-built before the demo (already done — the environment is standing): `setup/00_rbac.py` → `setup/01_load_data_prod.py` → `transforms/base_features.py` → `setup/02_feature_store.py` → `setup/03_train_register.py --version V1`.

Run live during the demo: the retrain loop (V2) and the promotion (via GitHub).

If you ever need to rebuild from scratch, run the five pre-build steps in order (README has them).

3. Order of operations (the live demo)

Act 1 — The sandbox boundary (~2 min) [UI or TERM]

Show the two databases and roles, then prove the boundary. In a Snowsight SQL worksheet:

```
USE ROLE ML_DEV_ROLE;
SELECT COUNT(*) FROM ML_FRAUD_PRODUCTION.CURATED.TXN_EVENTS;    -- works: dev reads prod
CREATE TABLE ML_FRAUD_PRODUCTION.FEATURE_STORE.X (a INT);      -- FAILS: dev cannot write prod
```

Say: dev has full *read* of prod (no masking needed — the point is process, not data). The enforced line is *deploy*. Only the service account can write prod, and only through CI.

Act 2 — The data scientist's first move (~3 min) [WS]

Open `demo/01_explore_and_prep.ipynb`. Run the EDA cells (fraud rate by channel), then the cell that calls `build_abt(session, "dev")`. **Say**: the DS doesn't build features off raw prod — they apply a **selective transform** into a clean, labeled **analytical base table (ABT)** in their sandbox. The *same* transform is promoted to prod later (dev/prod parity).

Act 3 — Feature store + training + experiment tracking (~3 min) [UI]

- **Feature Store** (AI & ML → Features): show ACCOUNT_PROFILE (+ ACCOUNT_RISK after Act 4).
- **Experiments** (AI & ML → Experiments → AML_FRAUD_TRAINING): show the TRAIN_V1 run's params + metrics.
- **Model Registry** (AI & ML → Models): show AML_FRAUD_GBM in the **dev** registry.

Say: training reads the ABT + feature store, logs every run's params/metrics for reproducibility, and registers the model to the *dev* registry.

Act 4 — The MLOps loop: add a feature, retrain, compare (~3 min) [WS]

Open `demo/02_add_feature_retrain.ipynb` in the Workspace and **run all cells**. It registers a **new** ACCOUNT_RISK feature view (spend volatility + receiver fan-out), retrains **V2**, logs a second experiment run, and prints the V1→V2 comparison (V2 wins). **Say**: this is the iteration loop — new signal → retrain → tracked comparison

→ V2 becomes the candidate (dev default). Refresh Experiments in [UI] to show TRAIN_V1 vs TRAIN_V2.

Act 5 — The promotion gate (~4 min) [GH]

This is the punchline. A data scientist **cannot** push V2 to prod. Go to **GitHub** → **Actions** → "**Promote model to production**" → **Run workflow** (input V2).

- GitHub mints an **OIDC token** — no key/secret stored.
- The production environment requires **your approval** (show the approval click).
- The workflow authenticates as SVC_ML_DEPLOY and **submits an ML Job** that runs the

promotion **server-side** on the compute pool: builds the prod ABT with the same transform, registers the prod feature store, promotes the model into the **prod** registry, and creates the batch scoring task.

Then in [UI]/[TERM] show the result:

```
SELECT COUNT(*) n,
       ROUND(AVG(IFF(LABEL=1, FRAUD_SCORE, NULL)), 3) avg_fraud,
       ROUND(AVG(IFF(LABEL=0, FRAUD_SCORE, NULL)), 3) avg_normal
FROM ML_FRAUD_PRODUCTION.ANALYTICS.PREDICTIONS;    -- ~0.91 vs ~0.13
```

Demo fallback: if GitHub connectivity/approval isn't ready, run the same promotion locally

from [TERM] — it exercises identical logic (runs the ML Job under ML_DEPLOY_SVC):

```
ML_ENV=dev .venv/bin/python demo/03_submit_promote_job.py --dev-version V2
```

4. Pitfalls & fixes

Symptom	Cause	Fix
Promotion ML Job takes minutes to start	Compute-pool / image cold start	Run demo_kickoff.py (--warm-image) beforehand
"compute pool busy (1/1 nodes)"	MLOPS_CPU_MAX_POOL is single-node; back-to-back jobs serialize	Only run one promotion at a time; or bump pool MAX_NODES=2 before the demo
GitHub Action can't auth (JWT subject/issuer not recognized)	The SVC_ML_DEPLOY OIDC subject doesn't match the token's sub, or production env / SNOWFLAKE_ACCOUNT variable is missing	Add the production environment + SNOWFLAKE_ACCOUNT / SNOWFLAKE_WAREHOUSE variables . Subject must equal the JWT sub exactly — this org issues immutable-ID subjects repo:<owner>@<owner_id>/<repo>@<repo_id>:environment:production, not the name-based form. Set OIDC_SUBJECT to that value when running 00_rbac.py (this account: repo:sfc-gh-tporter@69521549/snowflake-mlops-dev-sandbox@1277090492:environment:production)

GitHub Action IP blocked (390422)	Account VPN/IP network policy rejects the CI runner's dynamic IP	SVC_ML_DEPLOY gets a user-level allow-all policy (ML_DEPLOY_WIF_NETPOLICY) that overrides the account policy for just this WIF user — <code>00_rbac.py</code> creates and assigns it. Safe: the user can only auth via OIDC bound to the repo subject
ML Job fails with internal error (370001 / "incident")	Transient Snowflake server-side execution error — not your code/config (it got past auth, network, and started running SQL)	Re-run the workflow; it usually clears. Off-stage fallback that bypasses the ML Job entirely: <code>python demo/promote_model.py --dev-version V2</code>
Model load fails "owner only"	<code>mv.load()</code> needs the owner (dev) role as <i>primary</i>	Already handled — <code>promote_model.py</code> switches to <code>ML_DEV_ROLE</code> for load, back to <code>ML_DEPLOY_SVC</code> to write
"version V2 already existed"	Re-promoting an existing prod version	Already handled — promotion skips re-log if the prod version exists
Experiment run won't resume	A run name already ended	Already handled — scripts <code>delete_run</code> before <code>start_run</code>
Batch task didn't run	It's created suspended on purpose	For a live scored run: <code>EXECUTE TASK ML_FRAUD_PRODUCTION.ANALYTICS.SCORE_BATCH_TASK</code> ; or <code>ALTER TASK ... RESUME</code>
Online / Postgres cost worry	The optional online path (setup/online/) runs 24/7	Not used in this demo. Never run it unless showing real-time; teardown covers it

Note: batch scoring uses **no PAT** — the `SNOWFLAKE_PAT` is only for the optional online REST path.

5. Reset (run after the demo)

[TERM]

```
.venv/bin/python reset/teardown.py          # dry run (shows what it will drop)
.venv/bin/python reset/teardown.py --yes    # execute
```

Drops **both** databases, the roles, the OIDC service user, and the auth policy; leaves the shared compute pool. Includes an orphaned-Postgres safety net (for the optional online path). The demo currently stands with minimal idle cost (a few daily-lag Dynamic Tables; batch task suspended; no Postgres online) — so you can leave it up between rehearsals if you prefer.

6. Deep-dive talking points (explain it like a data scientist)

The dataset — IBM AML HI-Small (synthetic)

- ~5.08M transactions across ~518K accounts in a **multi-bank** ecosystem, with IS_LAUNDERING labels and a companion file of injected laundering **typologies** (fan-out, structuring, cycles).

- **Extreme class imbalance:** ~0.089% of transactions are laundering. This is realistic and it dictates everything about how we model and measure.

- **Synthetic on purpose:** the demo is about the *ML process*, not the data — so there's nothing sensitive, which is exactly why dev gets full read access with no masking.

- **Data hygiene:** we trim a trailing all-laundering tail ("Marco quirk") so a chronological train/test split can't leak the date as a predictor.

The ML problem & why we measure it this way

- Binary classification on a ~0.1% base rate. **Accuracy is useless** (predict "never fraud" = 99.9% accurate and catches nothing). We report **PR-AUC** and **recall@1%** (what share of fraud we catch in the top 1% of scored transactions — the alert budget an investigations team actually has).

- **Chronological split** (train on earlier, test on later) — no look-ahead leakage, mirrors production where you score the future.

The model

- **HistGradientBoostingClassifier** (scikit-learn). Chosen over XGBoost deliberately: no native libomp dependency, so it's portable and runs anywhere (including the ML Job runtime).

- **Class imbalance** handled with `sample_weight` (positives weighted ~100:1).

- **Monotonic constraints** (`monotonic_cst=+1`) on risk-increasing features: more amount / cross-border / cross-currency / velocity / fan-out must never *decrease* the risk score. This removes counterintuitive decision boundaries and makes the model behave sensibly for a probing audience.

- The output is an **uncalibrated relative risk score**, not a calibrated probability (we rebalanced training). Frame it as "rank/triage risk," not "P(fraud)=x".

The signals / features (three groups)

1. **Request-context** (per transaction, from the ABT): AMOUNT_PAID, IS_CROSS_CURRENCY, IS_CROSS_BORDER, IS_HIGH_RISK_FORMAT, AMOUNT_TO_AVG_RATIO, and one-hot PAYMENT_FORMAT.

2. **Account profile** (lifetime aggregates, ACCOUNT_PROFILE FV): average/stddev/max amount, distinct receivers / receiver-banks / countries, foreign-currency share, high-risk-format share.

3. **Engineered risk** (the V2 additions, ACCOUNT_RISK FV): HIST_AMOUNT_CV (spend volatility = std/avg) and HIST_RECEIVER_FANOUT (distinct receivers / txn count). **Fan-out is the classic money-laundering tell** — a mule spraying funds to many receivers — which is why adding it lifts the model.

- On "did it just learn ACH = fraud?": no. In this data ~99% of ACH is normal; the model learns *velocity + dispersion + new-account* patterns, and the same channel with an established profile scores low. (Feature importance / the V1→V2 delta backs this up.)

Why batch (default) and when online

- **Batch** = a scheduled task scores recent events into PREDICTIONS. No standing infrastructure → cheap, and it's how most AML monitoring actually runs (periodic scoring + case queue).
- **Online** (Postgres-backed feature store + real-time REST + SPCS) exists under setup/online/ for sub-second scoring, but runs 24/7 and costs money — so it's opt-in, not part of this demo.

The Snowflake ML Ops capabilities on display

- **Feature Store** — governed, reusable feature views (Dynamic Tables) over the ABT.
- **Model Registry** — versioned models, per-env (dev vs prod) registries, default version.
- **Experiment Tracking** — params + metrics per run, side-by-side comparison, reproducibility.
- **ML Jobs** — the promotion runs as a **server-side job on a compute pool** (operationalized, not a notebook); dependencies pinned via the job's `requirements.txt`.
- **Keyless CI/CD (OIDC / Workload Identity Federation)** — GitHub's OIDC token is validated by Snowflake directly; no key pair or secret stored. The modern replacement for key-pair service accounts.
- **RBAC governance** — the dev/prod boundary is enforced on *writes*: dev reads prod, only the service account deploys, and a GitHub environment approval adds a human gate.

7. Likely questions (quick answers)

- **"Is this real-time?"** Batch by default; the online/real-time path exists and is a config switch, but it carries 24/7 cost so we don't run it here.
- **"Can a data scientist just deploy?"** No — RBAC blocks dev from writing prod; only the OIDC service account can, and only through the approved GitHub workflow.
- **"Why synthetic data / no masking?"** The demo is about the ML process; the data isn't sensitive, which lets us show the (more interesting) *deploy* governance instead of masking.
- **"What did adding a feature actually buy?"** V1→V2: higher PR-AUC (~+0.03) and recall@1% (~+0.02) — shown live in the experiment comparison.
- **"What does promotion actually move?"** The exact trained artifact (loaded from dev, logged to prod), plus the same transform + feature definitions, plus the batch scoring task.

8. Command & file reference

Purpose	Command
Pre-warm	<code>ML_ENV=dev python pre_demo/kickoff.py [--warm-image]</code>
Build (fresh)	<code>00_rbac (dev) → 01_load_data_prod (prod) → transforms/base_features (dev) → 02_feature_store (dev) → 03_train_register --version V1 (dev)</code>

Live retrain	Open demo/02_add_feature_retrain.ipynb in the Workspace → Run all
Promote (GitHub)	Actions → "Promote model to production" → Run workflow (input V2) → approve
Promote (local fallback)	ML_ENV=dev python demo/03_submit_promote_job.py --dev-version V2
Reset	python reset/teardown.py --yes

Key objects: ML_FRAUD_PRODUCTION, ML_FRAUD_DEV_SANDBOX; roles ML_DEV_ROLE, ML_DEPLOY_SVC; service user SVC_ML_DEPLOY (OIDC); model AML_FRAUD_GBM; experiment AML_FRAUD_TRAINING; predictions ML_FRAUD_PRODUCTION.ANALYTICS.PREDICTIONS.

9. Objection handling — why things are pre-committed / pre-baked (the automation story)

This section exists so you can confidently answer the sharp MLOps questions ("isn't this too manual?", "features are code — where's the PR?", "why pull from the registry?"). Read it before presenting to an engineering audience.

The core framing (lead with this)

"Two different things flow through two different systems: **code** flows through Git (branch → PR → review → merge); **models** flow through the **Model Registry** as versioned artifacts. This pipeline promotes a *model*, so it pulls the artifact from the registry — it does not push a binary through Git."

That reframes every "why isn't X in Git" question: because X is a model artifact, and model binaries don't belong in Git — the registry is their system of record (versioned, governed, with metrics and lineage).

The two paths (say this explicitly)

- **Code changes** — a new feature definition, the transform, training or deployment logic →

branch → **PR** → **review** → **merge to main**. Standard software CI. This is where a new feature belongs.

- **Model changes** — same code, retrained on new data producing a better version → a new

registry version, promoted dev→prod by the pipeline you kick off. No Git involved.

The demo shows the **model/registry path + the governance gate** live, because that's the part people haven't seen; the code/PR path is normal CI.

"Features are code — isn't that being skipped?" (the honest answer)

Not skipped — **pre-committed**. The prod feature definitions (ACCOUNT_PROFILE, ACCOUNT_RISK) live in committed code in demo/promote_model.py (register_prod_fs()), and the Actions pipeline checks that file out of main and runs it. So the feature's path to prod is code → Git → pipeline.

What the demo simplifies: the notebook adds ACCOUNT_RISK **interactively** to show the iteration, and the matching prod definition is already committed — so there are two copies of the feature SQL (dev notebook + promote_model.py) that we keep in sync manually. The PR/review of that feature code is pre-baked rather than performed live.

"The feature reached production as code the pipeline pulled from main. On stage I add it in

the notebook to show the loop; the reviewed code change is pre-committed."

Why the promotion trigger is manual (workflow_dispatch)

It's a deliberate choice so the **gate is visible** on stage instead of firing invisibly. Wiring the trigger is the easy, last-mile change; in production you'd pick one of:

- **on: push to main** — a merged PR auto-kicks the promotion.
- **A registry event** — when a DS tags a version with an alias like Production-Candidate, a scheduled check fires the workflow via `repository_dispatch`.
- **A validation gate in the job** — auto-compare candidate vs current prod metrics, abort if worse.

None of that changes the security model — it only changes what pulls the trigger.

What IS automated / enforced (point here when pressed on "not automated enough")

- **No human can hand-deploy** — the dev role is blocked from writing prod (proven live in Act 1).
- **Keyless identity** — GitHub authenticates via OIDC/WIF; no stored key or password to leak.
- **Required approval** — the production environment gate needs a human sign-off.
- **Runs server-side as the service account (ML_DEPLOY_SVC)** on Snowflake compute, not a laptop.

"The automation that matters for governance — who can deploy, how they authenticate, that someone approved it — is fully enforced. That's the hard part, and it's built."

Why not perform the code PR/merge live? (the TLDR)

Three reasons it's pre-baked:

1. **It's generic.** PR → review → merge is vanilla software CI; the Snowflake-differentiated value (feature store, registry, keyless promotion, RBAC gate, ML Job) is what earns stage time.
2. **Live-failure risk.** A live PR + CI adds minutes and things that break on stage (merge conflicts, CI flakes, the OIDC 503). Pre-baking makes the demo deterministic.
3. **Stage time is scarce** — minutes in GitHub's PR UI aren't spent on the money moment.

Audience call: for **data/exec leadership**, pre-bake and keep it tight. For **MLOps/platform engineers**, consider doing a *minimal* live code change → PR → merge, since that's the exact objection they'll raise (their language, their concern).

How you'd harden it (the "production version" answer)

Factor the feature (and transform) definitions into **one shared, version-controlled module** that both the dev notebook and `promote_model.py` import. Then adding a feature is a real **PR → review → merge**, the notebook imports the identical definition, and dev/prod can't drift. That's exactly why the ABT transform lives in `transforms/base_features.py`; we inlined it in the notebook only for demo visibility — the module remains the prod source of truth. Features would follow the same pattern in a hardened build.

Drop-in one-liners

- "Code flows through Git; the model is an artifact and flows through the registry."
- "The feature code rides through Git in the committed promotion code; the PR step is pre-baked, not absent."
- "The trigger is manual so you can see the gate — wiring on: push or a registry event is one line of YAML."
- "It's not less automated — it's automating the right layer: the deploy mechanism and governance gate are enforced; the trigger is a config choice."

"How much of this is pre-built? Could I actually use it?" (reference implementation)

The worry: it looks like a lot is pre-baked (the transform, `promote_model.py`, RBAC, the pipeline) and the audience thinks "cool, but I could never run this in my pipeline." Reframe it: **a demo is a reference implementation, not a greenfield build — pre-built plumbing proves the pattern is real and repeatable, it's not a rigged trick.**

Split everything into two buckets and say it out loud:

- **Built once per project (the platform investment):** two-DB topology, RBAC boundary, OIDC identity, `transforms/base_features.py`, `promote_model.py`, the workflow YAML. A team writes these once and rarely touches them.
- **The repeating loop (weekly DS work):** explore -> add/adjust a feature -> retrain -> compare -> PR/promote.

You're demoing the **steady state**, not day zero. Nobody hand-writes Terraform live to prove a deploy is real.

- **Set the frame up front (Act 0):** "Everything here is in a public repo you can read and fork - the transform, the promotion job, the RBAC, the pipeline. I pre-built the plumbing a team sets up once, so we spend our time on the loop a data scientist runs constantly and the governance that makes it safe. Nothing is hidden."
- **Adoption pitch (kills "I couldn't use this"):** "Point it at your account, swap in your data and features, keep the RBAC + OIDC + ML-Job pattern as-is. The pattern transfers directly; only the model-specific code changes. It's a public repo - fork it."
- **You DO show an end:** a dev change -> gated keyless deploy -> model live in prod -> predictions landing. That is the ending; what's compressed is authoring the scaffolding (once) and the live PR trigger (audience-dependent).
- **Name what's not production-grade (reads as expertise):** a metric gate on promotion, an automated trigger, features as a shared module. "Those are the next hardening steps - the pattern already supports them."

"What if I have 10 models in prod - a script and job each?"

No. **One parameterized promotion engine + per-model config**, not N scripts.

- `promote_model.py` hardcodes one model for readability; the logic is generic ("load a dev version -> log to prod -> register its feature views -> set up scoring"). It already takes `--dev-version`; add `--model` and a manifest and it promotes any model.
- **Shared once:** the promotion mechanism, RBAC, OIDC identity, workflow, compute pool. **Per-model as config (not code):** model name, feature set, scoring target, metric thresholds, schedule - a `models.yaml` or a control table with one entry per model.
- **Features don't multiply either:** the feature store means 10 models *share* features rather than each redefining them.
- One-liner: "You don't write a script per model - you build one parameterized job and a manifest with an entry per model. The 11th model is a config entry and a PR, not a new pipeline."

10. Guided UI walkthrough (Snowsight) — make the platform visible

The live *typing* is thin (two notebooks + a promotion), but the *artifacts* are rich and mostly invisible. The demo lands when you **open the UI and show what was built** — before, between, and after the notebooks. This turns "we ran two notebooks" into "look at this whole governed ML platform." Nav labels are under the Snowsight left nav; they can vary slightly by version.

45-minute run-of-show (interleave notebooks + UI tours)

Time	Act	What you do	Surface
------	-----	-------------	---------

0:00–0:06	0. Environment tour	Set the stage: show the platform <i>before</i> touching it	Databases & schemas; quick peek at Features / Models / Experiments
0:06–0:10	1. Governance boundary	Notebook 01 boundary cell: read prod OK, write prod BLOCKED	Databases (dev vs prod)
0:10–0:15	2. Explore + build ABT	Notebook 01 rest: EDA + the inline transform → dev ABT	Data » dev ABT table
0:15–0:21	3. Baseline (pre-built V1)	Tour what setup produced	Feature Store, Model Registry (V1), Experiments (TRAIN_V1)
0:21–0:29	4. The loop	Notebook 02: add ACCOUNT_RISK, retrain V2, compare	Feature Store (new FV), Registry (V2), Experiments (V1 vs V2)
0:29–0:37	5. Promotion gate	Trigger + approve in GitHub; watch the job	GitHub Actions, then Query History
0:37–0:41	5b. Prod result	Show the model + predictions landed in prod	Model Registry (prod), Predictions + chart, Task
0:41–0:45	6. Recap + Q&A	Restate boundary / two-DB / keyless pipeline; objections	(section 9)

Databases & schemas — the dev/prod split (Act 0)

- **Where:** Data » Databases. Expand ML_FRAUD_PRODUCTION and ML_FRAUD_DEV_SANDBOX.
- **Show:** two databases in one account; prod has RAW / CURATED / FEATURE_STORE / ML / ANALYTICS, dev has its own CURATED / FEATURE_STORE / ML / EXPERIMENTS.
- **Say:** "One account, two databases. The data scientist owns the sandbox and can read all of prod — but as we'll see, cannot write it. The boundary is the whole point."

Feature Store — features as governed objects (Act 3 preview, Act 4 payoff)

- **Where:** AI & ML » Features. Select the feature store schema.
- **Show (Act 3):** the ACCOUNT entity and ACCOUNT_PROFILE feature view — its definition, refresh schedule, and the columns it serves. **Show (Act 4):** after retrain, the newly registered ACCOUNT_RISK view appears alongside it.
- **Say:** "Features are defined once here and reused for training and scoring — and the same definitions are promoted to prod, so dev and prod build features identically."

Model Registry — versions, metrics, governance (Act 3 + Act 4 + Act 5b)

- **Where:** AI & ML » Models » AML_FRAUD_GBM.
- **Show (Act 3):** in the **dev** registry, version V1 with its metrics (PR-AUC, ROC-AUC, recall@1%) and functions (PREDICT_PROBA). **Show (Act 4):** V2 appears, set as default; compare its metrics to V1. **Show (Act 5b):** switch to the **prod** registry — V2 is now there, promoted, default.

- **Say:** "The registry is the system of record for models — versioned, with metrics and lineage. Dev and prod are separate registries; promotion is a governed registry-to-registry move."

Experiments — the tracked before/after (Act 3 + Act 4)

- **Where:** AI & ML » Experiments » AML_FRAUD_TRAINING.
- **Show:** runs TRAIN_V1 and (after Act 4) TRAIN_V2; open the comparison to see params and metrics side by side.
- **Say:** "Every training run is logged — parameters and metrics — so the V1→V2 improvement is an auditable record, not a claim. This is the reproducibility layer."

GitHub Actions + Query History — the deploy, made concrete (Act 5)

- **Where (GitHub):** repo » Actions » "Promote model to production" » the run.
- **Show:** the **approval gate** (production environment), the **keyless OIDC** auth step (no secrets stored), and the **ML Job** submit + logs (PREDICTIONS populated: ... rows).
- **Where (Snowsight):** Monitoring » Query History (filter to recent).
- **Show:** the actual server-side statements the job ran — the model log and the native-SQL `<model>!PREDICT_PROBA( . . . ) batch-scoring CREATE TABLE.`
- **Say:** "No human hand-deploys. GitHub authenticates with a short-lived token, an approver signs off, and the work runs server-side as the service account. Here's the exact SQL it ran."

Tasks & Predictions — the recurring output (Act 5b)

- **Where (task):** Monitoring » Task History, or Data » ML_FRAUD_PRODUCTION » ANALYTICS » Tasks » SCORE_BATCH_TASK.
- **Show:** the scheduled daily task (created suspended), its definition (the scoring SQL).
- **Where (predictions):** open a worksheet and `SELECT * FROM ML_FRAUD_PRODUCTION.ANALYTICS.PREDICTIONS LIMIT 100;` then use the **Chart** toggle to plot FRAUD_SCORE, or run the fraud-vs-normal average to show separation (fraud ≈ 0.9 vs normal ≈ 0.13).
- **Say:** "Promotion runs once; the daily task keeps scoring with no human in the loop. Here are the predictions downstream systems consume — and the model clearly separates fraud from normal."

Presenter tips to fill the time without filler

- **Narrate the pre-built as if walking a new hire through the platform** — the setup did real work; the UI tour is where you take credit for it.
- **Open the UI right after the cell that produced the object** ("I just registered V2 — let's go see it") so cause and effect are obvious.
- **End on the predictions + governance recap**, not on the last notebook cell, so the closing beat is "governed platform → business output," not "a script finished."

Appendix B — Optional online / real-time branch

Only for showing real-time serving. It costs money (Postgres online store bills 24/7) and provisions slowly — never stand it up live. The core MLOps story lands fully on batch; this is an add-on flourish.

Where it lives (three pieces):

- `setup/online/setup_online.py` — the build: Postgres online feature store + streaming velocity FV.
- `pre_demo/enable_online.py` — **enable + verify**, run the **day before**.
- `demo/optional_online_realtime.ipynb` — the optional **Act 6** notebook.

Enable (day before) [TERM]:

```
ML_ENV=dev SNOWFLAKE_PAT=... .venv/bin/python pre_demo/enable_online.py --yes
```

Provisions the online service, polls until RUNNING (can take up to ~an hour), and does a smoke read. Leave it running until after the demo.

Notebook prerequisites (in the notebook's Service settings) [WS]:

- Container runtime; the online service RUNNING.
- Attach an **External Access Integration** + a **PAT secret**; set `PAT_SECRET` in the notebook

to that secret's normalized path.

Act 6 (optional) [WS]: open `demo/optional_online_realtime.ipynb`, Run all. It shows a **millisecond point lookup**, a live ingest burst with **sub-2s stream freshness**, and a **real-time score** off online-served features.

Teardown (mandatory to stop cost) [TERM]: `python reset/teardown.py --yes` drops the online service (with an orphaned-Postgres safety net).

Note: this notebook is a **scaffold** built from the proven real-time patterns; rehearse it once after enabling the online service, since it can't be validated until the service is up. The batch model scores off profile/risk features; the velocity FV here demonstrates the online store's streaming freshness (a fuller real-time model that consumes velocity is a further step).